Exercises on
Physical access structures – part 0

5 Oct. 2026

Sara Comai

Exercise 1

Query evaluation strategy

A table

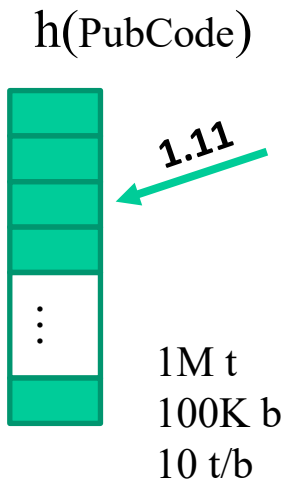
Publications(PubCode, ISBN, Title, Date, ...)

has ISBN as *candidate* key (unique, not null) and stores 1.000.000 tuples.
It is a hash-based structure, built on the *primary* key (PubCode), with:

- 10 tuples per block
- an average filling factor of 80%
- an average lookup cost of 1.11 i/o operations

***An idea of the table**:*

- 100.000 blocks in the primary hash structure (1M tuples, 10 tuples/block)
- The lookup costs of 1.11 is a given, «measured» figure, that cannot be derived from the other parameters, so we… «trust» the measurement! But how can we «visualize» it? We can «imagine» that there is 10% probability that a bucket is full and requires reading one extra «overflow» block, 1% probability that the overflow requires reading 2 extra blocks, negligible probability that a fourth block is needed.

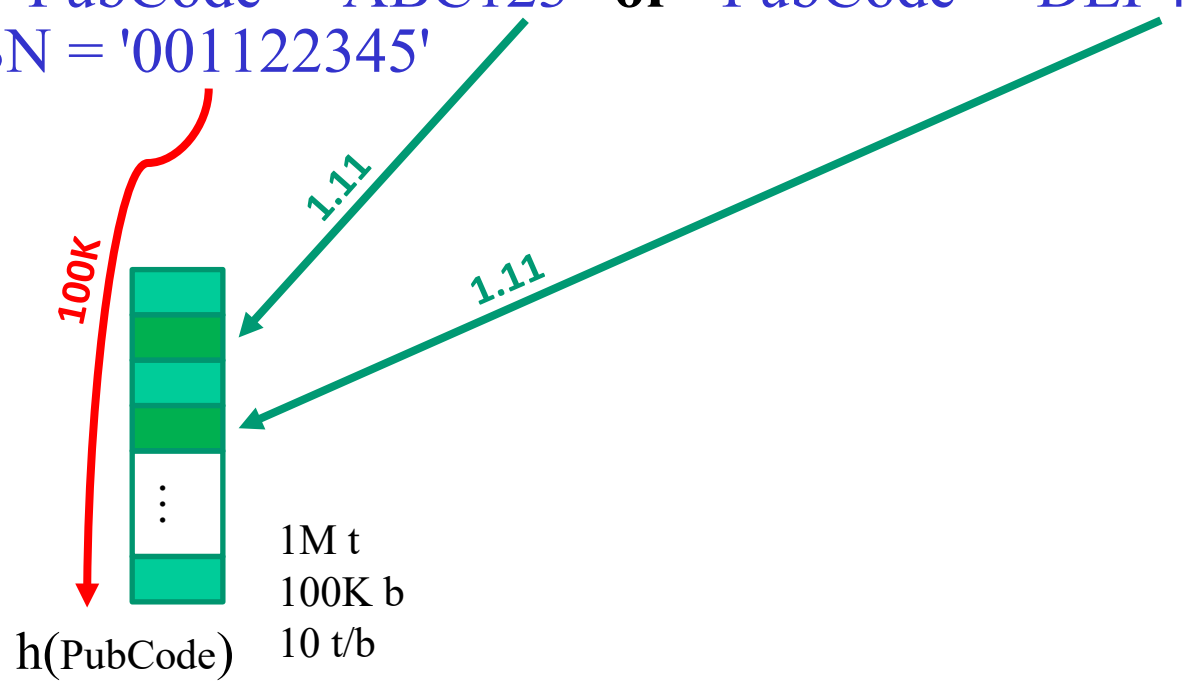


Query evaluation strategy

First Question (scenario 1)

a) Ignoring the possible effects of caching, and only taking account of disk i/o operations, estimate the execution cost of the query:

```
select *
from Publications
where   (   PubCode = 'ABC123'  or  PubCode = 'DEF456'   )
  and  ISBN = '001122345'
```



At the outmost level of the where clause there is a conjunction of predicates.

We have

- one access structure that supports indexed access based on one predicate (PubCode)
- no support for the other predicate (ISBN)

so only the *first conjunct* can be used to retrieve the relevant tuple.

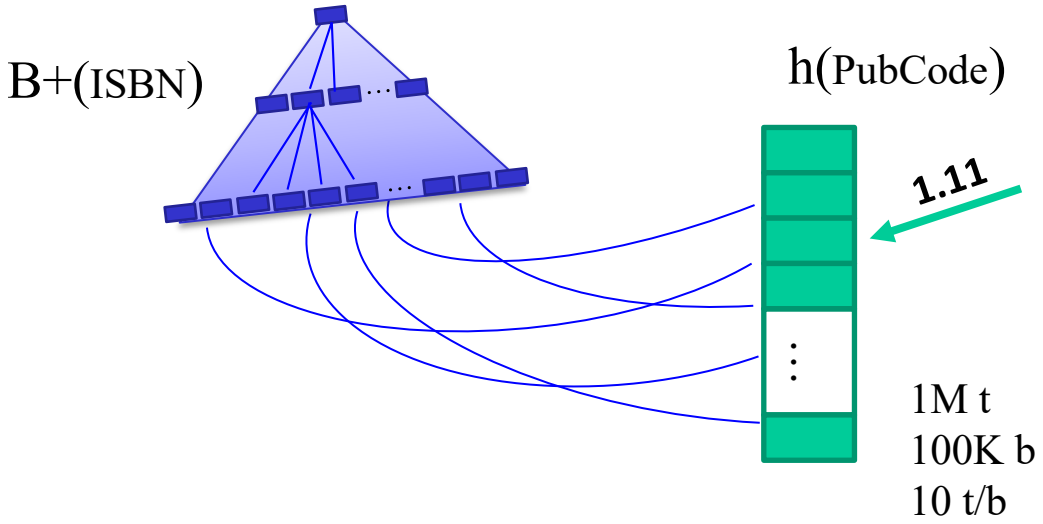
Being in turn a disjunction, the second conjunct requires 2 accesses to the structure to be fully evaluated, at the (average) cost of **2.22** i/o operations.

Second Question (scenario 2)

b) Assume that a secondary access structure is also available, in the form of a **B+ tree** index with ISBN as search key, with an average fan-out of 100.

Still ignoring the impact of caching mechanisms, estimate the cost of execution through the secondary structure and **compare the result with the previous one**.

select * from Publications
where (CodPubbl = 'ABC123' **or** CodPubbl = 'DEF456')
and ISBN = '001122345'



Even if (*optimistic assumption*) the B+ tree is full and well-balanced, its depth cannot be less than 3, because $\log_{100} 10^6 = 3$

Thus, each access to the index requires a minimum of 3 i/o ops just to reach a leaf node with the pointer to the primary storage (+1 access to actually retrieve the tuple).

We conclude that the most convenient approach is still the **previous strategy**.

Indeed, hash-based storage is hardly beatable for *random access*, whereas B+ trees excel with *interval queries*

Exercise 2

Try it at home

Consider the slides about indexes built on multiple attributes
(+ exploit statistics)

Choice of the most effective index

A table T(Name,Surname,City,Pathology,BloodType)
has 10.000.000 tuples and 3 multi-attribute indexes:

Idx**1**(City,Name,Surname)

Idx**2**(Pathlogy,Name,BloodType)

Idx**3**(Pathology,Surname,City)

The access keys are made by the listed attributes from left to right. All attributes have a (reasonably) uniform and independent distribution, with:

val(Name) = 1000 val(City) = 1000 val(Pathology) = 100
val(Surname) = 2000 val(BloodType) = 10

Choose the most convenient index to evaluate the query:

```
select *
from T
where Name = "N" and Surname = "S" and
      Pathology = "P" and BloodType = "BT"
```